

Reviewer Pack — Algebraic Hodge Index and Circuit Satisfiability Time

Entry 4876937d06d1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 01:53:01 UTC

Algebraic Hodge Index and Circuit Satisfiability Time

Entry ID: 4876937d06d1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 01:53:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (specifically, Hodge Theory) **Field B** (complexity object): Circuit Satisfiability Time

Statement:

For a given CNF formula F with n variables and m clauses, the number of satisfying assignments $t(F)$ for F is bounded by $O(H^2_F)$, where H^2_F is the second Betti number of the configuration space of the polynomial ring associated with F modulo 2.

Rationale (proposer's reasoning):

Hodge theory provides a way to measure the complexity of the geometry of complex algebraic varieties. By considering the configuration space of the polynomial ring associated with a CNF formula, we may be able to uncover structural properties that relate to the satisfiability time of the circuit.

Taxonomy category: ALGEBRAIC_HODGE_INDEX (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4ed137db84a5a5c2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the computed correlation coefficient between the second Betti number H^2_F and the number of satisfying assignments $t(F)$ for at least 95% of the generated CNF formulas F exceeds 0.7, AND no seed generates a CNF formula with $H^2_F > 20$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Algebraic Hodge index" AND "circuit satisfiability time" - "second Betti number" IN Polynomial Ring AND "CNF formula" - "Hodge Theory" AND "satisfying assignments bound"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=92.8s

5.1 Generated Python source

```
import random
from fractions import Fraction

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n) * (2 * random.choice([1, -1]) - 1) for _ in range(random.randint(1, n))]
        cnf.append(clause)
    return cnf

def matrix_multiplication(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    cols_B = len(B[0])
    result = [[sum(A[i][k] * B[k][j] for k in range(cols_A)) for j in range(cols_B)] for i in range(rows_A)]
    return result

def gaussian_elimination(matrix):
    rows = len(matrix)
    cols = len(matrix[0])
    rank = 0
    for i in range(min(rows, cols)):
        if matrix[i][i] == 0:
            swap_found = False
            for j in range(i + 1, rows):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    swap_found = True
                    break
            if not swap_found:
                continue
        factor = Fraction(matrix[i][i], 1)
        for j in range(i, cols):
```

```

        matrix[i][j] /= factor
    for j in range(rows):
        if j != i and matrix[j][i] != 0:
            factor = Fraction(matrix[j][i], 1)
            for k in range(i, cols):
                matrix[j][k] -= factor * matrix[i][k]
    rank += 1
return rank

def compute_second_betti_number(cnf):
    n = len(cnf)
    incidence_matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for i, clause in enumerate(cnf):
        for literal in clause:
            incidence_matrix[abs(literal)][i + 1] += 1
    return gaussian_elimination(incidence_matrix)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(n, n * (n - 1) // 2)
    cnf = generate_cnf(n, m)

    t_F = sum([2**n for _ in range(2**n)])
    H2_F = compute_second_betti_number(cnf)

    if H2_F > 20:
        return {
            "metric_name": "second_betti_number",
            "metric_value": H2_F,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": f"H^2_F too large: {H2_F}"
        }

    return {
        "metric_name": "second_betti_number",
        "metric_value": H2_F,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    if not sys.argv[1:]:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    else:
        seeds = list(map(int, sys.argv[1:]))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

if all(result["conjecture_holds"] for result in results):
    support_fraction = 1.0
else:
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

mean_value = sum(result["metric_value"] for result in results) / len(results)
std_value = (sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results)) ** 0.5

if support_fraction >= 0.95:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"H^2_F too large\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means the pre-registered support conditions could not be verified. | next: Re-run the test to ensure it completes successfully and verify if the pre-registered support conditions are met.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21973
2	preregistration	ollama_remote	glm4:latest	0	0	9495
3	novelty	ollama_remote	glm4:latest	0	0	8311
4	novelty	ollama_remote	glm4:latest	0	0	8880
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16486
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	142176
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31270
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20326
9	judge	ollama_remote	glm4:latest	0	0	138912

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 397828 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/4876937d06d1.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4876937d06d1.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4876937d06d1.tar.gz (if generated)